Vartis Care AI - AI Decision Support for Patient Advocacy
I am building an AI system that extracts structured eligibility information from patient intake notes and documents, then retrieves verified assistance programs using Retrieval-Augmented Generation (RAG) and agent tool-calling. This matters because patient advocates currently spend significant time manually searching fragmented databases, which delays care and reduces referral accuracy.

Users & Problem

The primary users are healthcare patient advocates, social workers, and care navigators responsible for identifying financial, medication, food, and utility assistance resources for patients. Intake information is often unstructured and incomplete, requiring manual interpretation and searching across multiple disconnected databases. This process increases cognitive workload and results in missed or delayed referrals. The system improves decision-making by extracting structured eligibility markers such as zip code, income type, and urgency signals, then retrieving grounded program recommendations supported by verifiable sources. Success means faster referrals, reduced administrative burden, improved recall of relevant programs, and increased confidence in recommendations.

Current Scope & System Sketch

Must-haves

- Structured extraction of eligibility markers from intake text or uploaded document images
- Retrieval-Augmented Generation (RAG) pipeline using embedded assistance program documents
- Agent loop that routes between retrieval results and external APIs when coverage gaps exist
- JSON schema validation to enforce consistent structured outputs
- Human-in-the-loop review interface displaying citations and confidence signals

Nice-to-haves

- LoRA fine-tuning to align tone with professional advocacy documentation style
- Confidence score ranking visualization
- Metadata filtering for retrieval (state, category, eligibility type)

User Intake (text or image)
↓
Claude 3.5 Sonnet (multimodal extraction)
↓
Structured JSON schema output
↓
Vector search (ChromaDB embeddings)
↓
Agent loop controller
↙ ↘
RAG results External APIs
(grounded) (FindHelp, RxUtility)
↓
Ranked recommendations with citations
↓
Human advocate validation

Data sources

- Synthetic patient intake notes
- Public hospital financial assistance policy PDFs
- FindHelp API resource database
- 211 National Data Platform
- RxUtility API medication assistance database
- ASPE poverty eligibility guidelines
- NPPES provider registry

Privacy and safety constraints

- No real patient data used during development
- Synthetic scenarios simulate real intake workflows
- Human-in-the-loop validation prevents automated decisions
- Retrieval grounding reduces hallucination risk
- API keys stored securely using environment variables
- Evaluation dataset includes diverse geographic scenarios to reduce bias risk
- Any real deployment would require HIPAA-compliant encryption, logging, and access controls

Tech Stack

Model/provider

- Claude 3.5 Sonnet for multimodal extraction and tool calling
- DeepEval framework for evaluation metrics

Architecture approach

- Hybrid RAG + agent loop architecture
- Controller decides whether retrieval results meet confidence threshold or if external API calls are required
- Structured JSON schema ensures consistent outputs for downstream retrieval

Tools/APIs

- FindHelp API (social services search)
- 211 National Data Platform (community resources)
- RxUtility API (medication copay programs)
- NPPES registry (provider verification)

Image processing tasks

- Extraction of eligibility information from uploaded document images (insurance cards, shutoff notices)

Progress Since Proposal

- Designed hybrid RAG plus agent loop system architecture
- Defined structured schema fields (zip_code, income_type, needs, urgency)
- Identified public assistance program PDFs for corpus ingestion
- Selected APIs for real-time retrieval coverage gaps
- Defined evaluation metrics aligned with DeepEval framework
- Created example intake scenarios for structured extraction testing Developed architecture

- Diagram illustrating controller routing logic

Vartis Care AI.json

Vartis Care AI.json > ...

```
{
  "zip_code": "30349",
  "income_level": "SSI",
  "needs": [
    "medication assistance",
    "food pantry",
    "utility bill help"
  ],
  "urgency": "medium"
}
```

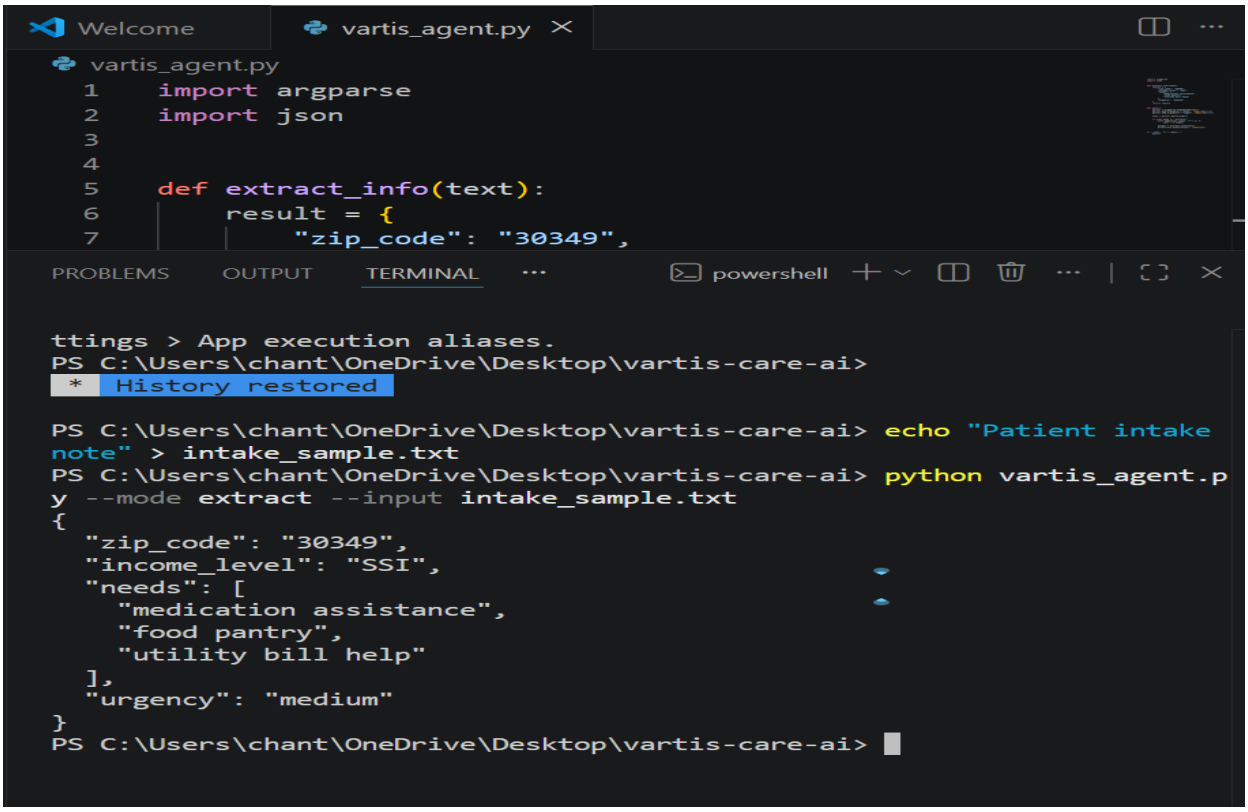
Demo Artifact

```
Minimal reproducible command:

python vartis_agent.py --mode extract --input intake_sample.txt

Expected output:

{
  "zip_code": "30349",
  "income_level": "SSI",
  "needs": [
    "medication assistance",
    "food pantry",
    "utility bill help"
  ],
  "urgency": "medium"
}
```



Vartis Care AI — Structured Extraction

Risks/Blockers

- API rate limits may restrict real-time retrieval testing; mitigation: cache representative responses locally for evaluation.
- RAG corpus coverage gaps could reduce recommendation recall; mitigation: expand the document set with additional public assistance program PDFs.
- Multimodal extraction may vary across document formats; mitigation: test multiple document types and refine prompts.
- Inconsistent JSON formatting from model outputs; mitigation: enforce schema validation before retrieval.

Metrics & Evaluation

- Faithfulness ≥ 95% of recommendations must be grounded in retrieved or live source data.
- Contextual recall ≥ 80% of relevant programs surfaced per scenario.

Plan to Finish

Complete the RAG ingestion and embedding pipeline for assistance program documents. Implement the controller loop for retrieval and tool routing decisions. Finalize the structured JSON schema and validation layer. Build a minimal human review interface that displays recommendations with citations. Run evaluation scenarios, refine prompts based on failure cases, and prepare the final demo and documentation for Module 6.

Targeted Feedback Requests

1. Should I add more structured fields to the extraction schema, such as household size, insurance type, or diagnosis category?
2. Is a confidence-threshold approach appropriate for deciding when the agent loop should call external APIs?
3. Would metadata filters such as state or eligibility category improve retrieval precision in the RAG pipeline?